

CI Integration with Artifact Repositories

3 hands-on labs: secrets handling, a working GitHub Actions pipeline, and enforcing CI best practices

LABS IN THIS GUIDE

- Lab 1 — Configure Maven authentication for CI using secrets, not hardcoded credentials
- Lab 2 — Build a GitHub Actions pipeline that resolves, builds, and publishes to Nexus
- Lab 3 — Enforce versioning & fail-fast best practices in the pipeline

Student lab guide · Companion to "CI Integration with Artifact Repositories" (Part 3 of 3)

1 Lab 1: Secure Authentication for CI

Objective: Configure Maven to authenticate against Nexus using environment-variable-driven credentials, and prove that no secret is ever committed to source control.

Prerequisites

- Completed the Nexus lab guide (Part 2), with `maven-releases` / `maven-snapshots` running
- A GitHub repository for the Maven project (can be a fresh empty repo)

Steps

Step 1 — Write settings.xml using environment variable placeholders

```
<server>
  <id>nexus-releases</id>
  <username>${env.NEXUS_USER}</username>
  <password>${env.NEXUS_PASS}</password>
</server>
```

Save this as `ci-settings.xml` inside the project repo (this file is safe to commit — it contains no actual secret values).

Step 2 — Add repository secrets in GitHub

In the GitHub repo: **Settings > Secrets and variables > Actions > New repository secret**. Add:

- `NEXUS_USER`
- `NEXUS_PASS`

Step 3 — Verify locally with environment variables (never hardcoded)

```
export NEXUS_USER=admin
export NEXUS_PASS=your_password
mvn -s ci-settings.xml deploy
```

Step 4 — Confirm nothing sensitive is in git history

```
git log -p -- ci-settings.xml | grep -i password
```

Confirm this returns nothing — only the `${env.NEXUS_PASS}` placeholder should ever appear in the committed file.

Verification Checklist

- [] `ci-settings.xml` is committed to the repo with placeholders only
- [] Real credentials exist only as GitHub Actions secrets, never in code
- [] `mvn -s ci-settings.xml deploy` succeeds locally using exported env vars

Discussion question: What would happen if `NEXUS_PASS` were hardcoded into `pom.xml` and later the repo was made public? Discuss credential rotation as a mitigation.

2 Lab 2: Build the Full CI Pipeline

Objective: Build a working GitHub Actions workflow that reproduces the full flow from the source document: checkout → resolve → build & test → package → publish to Nexus.

Prerequisites

- Completed Lab 1 (secrets configured)

Steps

Step 1 — Create the workflow file

Create `.github/workflows/ci.yml`:

```
name: CI Build and Publish

on:
  push:
    branches: [main]

jobs:
  build-and-publish:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Build and test
```

```

run: mvn -s ci-settings.xml clean verify
env:
  NEXUS_USER: ${ secrets.NEXUS_USER }
  NEXUS_PASS: ${ secrets.NEXUS_PASS }

- name: Publish to Nexus
  run: mvn -s ci-settings.xml deploy -DskipTests
  env:
    NEXUS_USER: ${ secrets.NEXUS_USER }
    NEXUS_PASS: ${ secrets.NEXUS_PASS }

```

Step 2 — Push and observe the run

```

git add .github/workflows/ci.yml ci-settings.xml
git commit -m "Add CI pipeline"
git push origin main

```

Open the **Actions** tab in GitHub and watch the workflow execute step by step.

Step 3 — Map each workflow step back to the source document's flow

On paper or in a shared doc, have students label each YAML step with the corresponding stage from the 6-step flow in the source document (checkout, resolve, compile & test, package, publish, downstream consumption).

Step 4 — Confirm the artifact landed in Nexus

Check the Nexus UI for the new version under `maven-releases` or `maven-snapshots`, matching the version declared in `pom.xml`.

Verification Checklist

- [] Workflow runs successfully end-to-end on push to `main`
- [] Published artifact is visible in the correct Nexus repository
- [] Student can correctly map each YAML step to a stage in the CI flow diagram

Stretch goal: Split the workflow into two jobs — one that runs on pull requests (build & test only, no publish) and one that runs only on merge to `main` (build, test, and publish).

3 Lab 3: Enforce CI Best Practices

Objective: Apply the four best practices from the source document to the pipeline built in Lab 2: consistent versioning, fail-fast dependency resolution, no public-repo fallback, and CI-runner-level caching.

Prerequisites

- Completed Labs 1 & 2

Steps

Step 1 — Enforce fail-fast dependency resolution

Add the offline-safe resolution flag so the build fails immediately if a dependency can't be found in Nexus, instead of silently reaching out to the public internet:

```
mvn -s ci-settings.xml -o clean verify
```

Temporarily rename the Nexus mirror URL to something invalid in `ci-settings.xml`, re-run the workflow, and confirm the build **fails loudly** rather than falling back to Maven Central.

Restore the correct URL afterward.

Step 2 — Add semantic versioning enforcement

Add a quick shell check as a workflow step, before the deploy step, that rejects a build if the version doesn't follow `MAJOR.MINOR.PATCH` or end in `-SNAPSHOT`:

```
- name: Validate version format
  run: |
    VERSION=$(mvn -s ci-settings.xml help:evaluate -Dexpression=project.version -q -DforceStdout)
    echo "Version: $VERSION"
    if ! [[ "$VERSION" =~ ^[0-9]+\.[0-9]+\.[0-9]+(-SNAPSHOT)?$ ]]; then
      echo "Invalid version format: $VERSION"
      exit 1
    fi
```

Step 3 — Add CI-runner-level dependency caching

```
- name: Cache Maven dependencies
  uses: actions/cache@v4
  with:
    path: ~/.m2/repository
```

```
key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}
restore-keys: ${{ runner.os }}-m2-
```

Place this step immediately after checkout, before the build step. Re-run the pipeline twice and compare build duration between the first (cold) and second (warm cache) run.

Step 4 — Confirm credentials still never appear in logs

Review the Actions run logs and confirm `NEXUS_PASS` is masked as `***` wherever it would otherwise be printed.

Verification Checklist

- ☐ Build fails clearly when Nexus is unreachable, instead of silently using the public internet
- ☐ Pipeline rejects an invalid version string before attempting to publish
- ☐ Second (cached) pipeline run is measurably faster than the first
- ☐ No secret values appear unmasked anywhere in the workflow logs

Wrap-up discussion: Walk through the "big picture" line from the source document — Maven/Gradle handle the "how," Nexus handles the "where," and CI is the glue. Ask each student to point to the exact pipeline step in Lab 2/3 that represents each part of that sentence.

Lab Guide — companion to "CI Integration with Artifact Repositories" (Part 3 of 3).